

[별첨 2]

AI Bot Olymbits

개발 활동 보고서

— CodinGame Summer Challenge 2024 · 한국 1 위 —

개발자	박진우 (goodpju2008@gmail.com)
플랫폼	CodinGame — 글로벌 코딩 및 AI Bot 경진 플랫폼
종목	Olymbits — Summer Challenge 2024 (CodinGame 연례 글로벌 챌린지)
구현 언어	C++
소스코드	source_ai_bot_olymbits/olymbits.cpp
공식 리더보드	https://www.codingame.com/multiplayer/bot-programming/summer-challenge-2024-olymbits/leaderboard
활동기간	2024
작성일	2026-05-12

1. 개요

1.1 보고서 목적

이 보고서는 CodinGame이 2024년 여름에 개최한 글로벌 챌린지 Olymbits (Summer Challenge 2024) 종목에서 동작하는 AI Bot에 대해, 제가 어떤 설계 결정을 내렸고 어떤 알고리즘 구조를 적용했으며 실제 성능 결과는 어땠는지를 정리한 개발 활동 보고서입니다. 이 봇은 전 세계 6,332명이 참가한 같은 종목에서 글로벌 175위(상위 2.8%), 한국 1위를 기록했습니다.

1.2 한 줄 요약

Expectimax + α - β 가지치기(깊이 12)를 기반으로, Sigmoid 기반 상대 행동 확률 모델링과 Log-기댓값 변환을 통한 다목적 → 단일목적 변환을 결합한 의사결정 AI입니다. 4개 미니게임이 동시에 진행되고 3명의 플레이어가 동시에 행동하는 복합 환경에서, 100ms 시간 제약 안에 자율적으로 최적의 행동을 결정합니다.

2. 개발 배경 및 문제 정의

2.1 종목 룰

Olymbits는 일반적인 봇 대결 종목과는 차원이 다른 복합 동시-다목적 게임입니다.

- **4 개의 미니게임을 동시 진행:** 허들 레이스(Hurdle Race), 양궁(Archery), 스케이트(Roller Speed Skating), 다이빙(Diving)
- **3 명의 플레이어가 동시에 한 가지 행동(LEFT / UP / RIGHT / DOWN)을 선택**
- 선택한 한 가지 행동이 **4 개 미니게임 모두에 동시 작용** (예: RIGHT 를 누르면 허들에서는 오른쪽 이동, 양궁에서는 조준 이동 등 동시에 적용됩니다)
- 최종 점수 = **4 개 종목 메달 가치의 곱(Multiplicative Score)**
- 시간 제약: 첫 턴 1 초, 이후 매 턴 100ms 이내

2.2 AI 관점의 도전 과제

특성	결과	함의
행동 공간	4 (LEFT/UP/RIGHT/DOWN)	작아 보이지만 동시-동시-동시 적용
다목적성	4개 게임 동시 평가 필요	단일 목적함수로 환원 필요
동시 행동	3명 동시 결정	표준 미니맥스 부적합
점수 결합 방식	메달 가치의 곱	한 종목 0이면 전체 0 (발산 위험)
각 게임의 상태 차원	게임마다 별도(위치/위험도/스턴 등)	통합 평가함수 설계 난이도 ↑
시간 예산	100ms / 턴	12레벨 탐색을 위해선 분기 축소 필수

이 종목은 단순 게임 트리 탐색으로는 한계가 분명해합니다. (1) 동시-행동 게임의 확률적 상대 모델링, (2) 다목적 → 단일목적 변환, (3) 게임별 도메인 지식 평가함수, (4) 알파-베타 효율 극 대화의 4가지 요소를 모두 결합해야 한다고 판단했습니다.

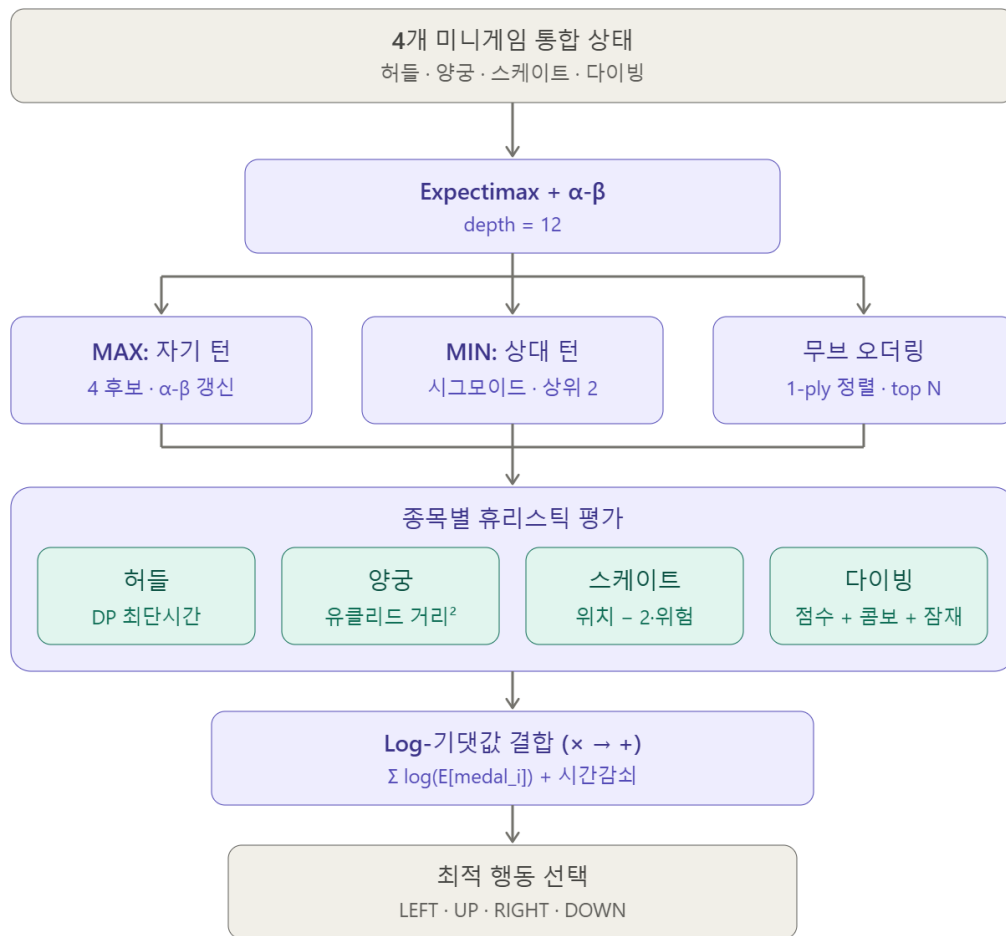
2.3 개발 목표

목표	정량 지표	결과
100ms 안에 깊이 12 게임 트리 탐색 수행	depth ≥ 12 Expectimax	✓ 달성
동시 행동 게임의 상대 모델링	Sigmoid 정책으로 확률화	✓ 달성
급셈 목적함수의 α - β 가지치기 적용 가능	Log-기댓값 변환	✓ 달성
4개 미니게임 각각에 도메인 특화 평가	게임별 휴리스틱	✓ 달성
글로벌 상위권 진입	Top 5% 이내	✓ 달성 (Top 2.8%)
한국 내 최고 성능	한국 1위	✓ 달성

3. 시스템 아키텍처

3.1 전체 데이터 흐름

아래 다이어그램은 매 턴 의사결정이 어떤 순서로 진행되는지 전체 진행 과정의 흐름을 나타냅니다.



3.2 핵심 설계 결정

결정	근거
Expectimax + α - β 를 주 알고리즘으로 선택	봇이 3개인 게임이라 최적/최악의 수만 고려하는 Minimax 부적합, 확률적 상대 모델링 필요
Sigmoid 기반 상대 행동 확률 모델링	"상대도 가능한 좋은 수를 둔다"는 Boltzmann 정책 — RL의 Soft Policy와 동치
Log-기댓값 변환	메달 점수가 곱셈 결합 → 작은 종목이 0이면 전체 발산. 로그 합산으로 변환해 α - β 적용 가능 + 기하평균 효과
게임별 휴리스틱 분리 설계	각 미니게임의 상태 구조와 보상 구조가 완전히 다름
무브 오더링 + 빔 폭 동적 조정	깊이 12 탐색을 100ms에 맞추기 위한 가지치기 효율 극대화

Undo 스택 기반 상태 복원	매 노드 깊은 복사 회피, 12레벨 탐색에 필수
온도 게임별 튜닝	4개 게임의 코스트 스케일이 다름 (허들 3, 양궁 150, 스케이트 8, 다이빙 20)
SIMD/AVX2 컴파일 옵션	하루프 자동 벡터화

4. 핵심 알고리즘 설계

4.1 핵심 알고리즘 ① — Expectimax + α - β 가지치기

표준 미니맥스는 상대를 결정론적 최적 플레이어로 가정하는데, Olymbits는 3명이 동시에 행동하는 게임이라 결정론적 상대 모델이 부정확합니다. 그래서 Expectimax 변형을 사용했습니다.

- 코드: olymbits.cpp:409-531
- **MAX 노드(자기 턴)**: 4개 후보 수를 모두 평가해 최댓값 선택 + α - β 가지치기 (olymbits.cpp:447-464)
- **MIN 노드(상대 턴)**: 상대가 비례적으로 합리적인 행동을 한다고 가정 → 상대 평가함수 기준 상위 2개 수에 대해 최솟값 (olymbits.cpp:506-529)
- 최대 깊이 12 레벨까지 탐색 (mxdep=12, olymbits.cpp:8) — 한 턴 = 3개의 결정 노드이므로 실질적으로 4턴 앞을 내다봅니다.

4.2 핵심 알고리즘 ② — Sigmoid 기반 상대 행동 확률 모델링 (★ 차별점)

순수 미니맥스가 부적합한 동시 행동 게임에서, "상대가 자기에겐 유리한 수를 둘 확률"을 시그모이드로 모델링했습니다.

$$P(\text{상대 A가 상대 B를 이길 확률}) = \sigma((\text{cost_A} - \text{cost_B}) / T) \\ = 1 / (1 + \exp(-(\text{cost_A} - \text{cost_B})/T))$$

- 시그모이드: olymbits.cpp:51-53
- 각 미니게임마다 별도 구현(prob 함수): 허들 63-65 / 양궁 153-155 / 스케이트 201-203 / 다이빙 363-365

온도(Temperature)는 게임별로 별도 튜닝했습니다:

게임	온도 T	튜닝 근거
허들	3	이산적 위치 차이 — 작은 스케일
양궁	150	거리 제곱 단위라 스케일이 매우 큼
스케이트	8	위치 + 위험도 + 스텝 — 중간 스케일
다이빙	20	점수 + 잠재력 — 누적 점수라 큰 스케일

시그모이드($\sigma(x) = 1/(1+e^{-x})$)는 Boltzmann 분포의 2-action 특수형이고, 강화학습의 Soft Policy(Soft Actor-Critic 등)와 동치입니다. 즉 상대를 "결정론적이 아니라 비례적으로 합리적"으로 모델링한 것이고, 동시 행동 게임에서 실제 인간/봇 상대의 거동에 더 가깝다고 생각합니다.

4.3 핵심 알고리즘 ③ — Log-기댓값 변환 (★ 차별점)

Olympbits의 최종 점수는 4종목 메달 가치의 곱입니다. 곱셈 형태는 α - β 가지치기와 잘 맞지 않고, 작은 메달이 0이면 전체가 0이 되는 발산 위험도 있습니다. 그래서 다음과 같이 변환했습니다.

```
maximize  $\prod E[\text{medal}_i]$ 
 $\equiv$  maximize  $\log \prod E[\text{medal}_i] = \sum \log E[\text{medal}_i]$ 
```

- 각 종목별 `score()` 함수가 반환하는 값이 이미 $\log(\dots)$ (olymbits.cpp:68-74)
- 합산된 로그 기댓값을 평가함수로 사용 (olymbits.cpp:411)
- 결과적으로 **로그-기댓값 최대화 $\equiv$ 기하평균 최대화** — 한 종목이 망해서 0이 되는 위험을 자연스럽게 피하고, 4 종목에 균형 있게 메달이 분배되도록 유도됩니다.

아래는 실제 코드 발췌입니다 (olymbits.cpp L411):

```
ld ret = hud.score(me,0) + arc.score(me,1)
      + skt.score(me,2) + divv.score(me,3)
      - ( hud.score((me+1)%3,0)+arc.score((me+1)%3,1)+...
          + hud.score((me+2)%3,0)+arc.score((me+2)%3,1)+... ) / 2;
// 자기 로그-기댓값 - 상대 두 명 평균 로그-기댓값
```

5. 구현 상세

5.1 무브 오더링 (Move Ordering) — α - β 효율 극대화

α - β 가지치기는 좋은 수를 먼저 평가할수록 가지치기 효과가 커집니다. 제 봇은 깊이마다 1-ply 예비 평가 → 정렬 → 깊은 탐색 패턴을 사용합니다.

1. 4 개의 모든 수를 일단 1-ply 깊이로 평가 (olymbits.cpp:436-446)
2. 점수 내림차순 정렬 (sort(lst, lst+4, greater<pli>()), olymbits.cpp:447)
3. 깊이가 깊어질수록 후보 폭을 줄임: $4 - \min(2, \text{depth}/3)$ (olymbits.cpp:448) — 빔 서치(Beam Search) 변형

깊이	탐색 후보 수
0~2	4개 (모두)
3~5	3개
6+	2개

이렇게 하면 트리 폭이 지수적으로 폭발하는 것을 막으면서, 좋은 수에 시간을 집중할 수 있습니다.

5.2 게임별 휴리스틱 평가함수 설계

각 미니게임마다 도메인 지식 기반의 코스트 함수를 따로 설계했습니다.

5.2.1 허들 (Hurdle Race)

- DP 로 골인까지의 최단 시간 사전 계산 (olymbits.cpp:122-139)
- $dp[i]$ = 위치 i 에서 골인까지의 최소 비용 (장애물 '#'을 만나면 +4 페널티 반영)
- 코스트: $-dp[position] - \text{stun_timer}$ → 골인 가까울수록, 스톤 적을수록 우위

5.2.2 양궁 (Archery)

- 표적 중심까지의 유클리드 제곱거리 ($\sqrt{\text{생략}}$ 으로 가속): $\text{cost} = -(x^2 + y^2)$
- 단조 변환이므로 거리 자체를 쓰는 것과 동등하지만 $\sqrt{\text{호출}}$ 1 회 제거

5.2.3 스케이트 (Roller Speed Skating)

- 선형 결합 평가: $position - 2 \times risk - 2 \times stun_timer$
- 위험도와 스톤 가중치를 위치 가치의 2 배로 페널티 부여
- 깊이 3 이상에서는 랜덤 시뮬레이션으로 전환 (olymbits.cpp:224-249) — 행동의 효과가 다음 턴 상태에 의존하므로 깊은 탐색에서는 결정론보다 기댓값 추정이 정확

5.2.4 다이빙 (Diving)

- 콤보 기댓값 + 남은 턴 잠재력: $point + combo \times (\text{남은 턴})/2 + \text{남은 턴} \times (\text{남은 턴} + 1)/2$
- 현재 점수 + 콤보 유지 시 예상 추가 점수 + 남은 턴 동안의 등차수열 합으로 잠재력 평가
- 콤보가 끊기지 않도록 행동 선택을 유도

5.3 동적 메달 가치 (Temporal Discounting)

이미 자신/상대가 딴 메달은 시간이 지날수록 가치를 감쇠시켰습니다.

```
medalvalue[t][i] = 1/4 * (turns3) + 3 * gold + 1 * silver + (-0.001) * bronze
```

- 코드: olymbits.cpp:550-556
- 게임 후반으로 갈수록 기본 보너스가 감쇠하여, 남은 게임의 결과에 더 큰 가중치가 실리도록 설계
- 동(bronze) 보상을 -0.001 로 매우 작게 줘서 동메달보다는 은메달 이상을 노리는 전략을 학습
- 강화학습의 할인계수(γ)와 보상 설계(reward shaping)에 대응되는 개념

5.4 상태 되돌리기(Undo Stack) — 트리 탐색 메모리 최적화

12레벨 깊이 탐색에서 매 노드마다 4종목 상태를 깊은 복사하면 메모리·시간이 폭발해버립니다. 그래서 변경분만 스택에 기록하는 방식으로 풀었습니다.

- *trackint*, *trackpii*, *tracktuple* 커스텀 스택 (olymbits.cpp:13-50)
- 각 게임의 `move()`는 `push`, `undo()`는 `pop` 으로 O(1) 상태 복원
- `std::vector` 대신 정적 배열 기반 스택으로 캐시 효율 극대화

5.5 멀티 에이전트 동시 행동 모델링

Olymbits는 3명이 동시에 행동하지만, 표준 게임 트리는 순차 행동을 가정합니다. 동시 행동을 다음과 같이 순차 행동으로 환원했습니다.

- `expectimax(depth, turn)`에서 `turn` 을 $0 \rightarrow 1 \rightarrow 2$ 로 순환 (olymbits.cpp:466)
- 세 번째 플레이어 차례가 끝났을 때만 게임 시간 진행 (`curtime++`, `turnsleft--`, olymbits.cpp:511-515)
- 즉 한 게임 턴 = 3 개의 결정 노드로 펼쳐 동시 행동을 순차 행동으로 변환 (Sequential equivalent of simultaneous game)

5.6 저수준 성능 최적화

최적화	코드 위치	효과
<code>#pragma GCC optimize("Ofast,unroll-loops,inline")</code>	L2	컴파일러 최적화 최대
<code>#pragma GCC target("avx2,fma,sse,...")</code>	L3	SIMD 명령어(AVX2/FMA) 활성화
정적 배열 기반 Undo 스택	L13-50	캐시 효율 + 동적 할당 0
<code>inline</code> 키워드 + 작은 함수	전체	하트로 인라이닝
√ 호출 제거 (거리?)	L150-152	양궁 평가 1회당 √ 호출 0번

5.7 적용 AI 기법 총정리

#	기법	AI 분야 분류
1	Expectimax + α - β 가지치기 (깊이 12)	확률적 게임 트리 탐색
2	Boltzmann/Softmax 정책 (Sigmoid)로 상대 모델링	베이지안/확률적 상대 모델 (RL Soft Policy)
3	온도 파라미터 게임별 튜닝	하이퍼파라미터 최적화
4	Log-기댓값 변환 (곱셈→합산)	다목적 → 단일목적 변환 (기하평균 최적화)

5	무브 오더링 + 빔 서치 (4 - min(2, depth/3))	α - β 최적화 / 빔 탐색
6	게임별 휴리스틱 평가함수 (DP, 거리 ² , 선형결합, 잠재력)	도메인 지식 임베딩
7	시간 감쇠 메달 가치 (Temporal Discounting)	RL 보상 설계 / Reward Shaping
8	동시-행동 → 순차 결정 환원	멀티 에이전트 모델링
9	Undo 스택 기반 상태 복원	탐색 메모리 최적화
10	SIMD/AVX2 컴파일 최적화	하드웨어 가속

6. 성능 측정 결과

6.1 글로벌 리더보드 성적

Olymbits - Summer Challenge 2024					
6,332 CodinGamers in the Olymbits - Summer Challenge 2024 arena					
DETAILS LEADERBOARD DISCUSSIONS					
World	Search for a friend, school, ...	League	CodinGamer	Language	Score
				School / Company	Country
	175	goodpjw2008	C++	21.50	Korea Digital Media High School - Ansan-si
	1	MSz	C++	32.45	University of Wrocław / University of Wrocław
	2	reCurse	C++	32.13	N/A

[그림] Olymbits 리더보드 — 한국 1위

공식 리더보드(증빙): <https://www.codingame.com/multiplayer/bot-programming/summer-challenge-2024-olymbits/leaderboard>

리더보드 캡처에서 제 계정 goodpjw2008이 한국 부문 최상단에 있는 것을 확인할 수 있습니다. Summer Challenge 2024는 CodinGame이 매년 여름 개최하는 연례 글로벌 챌린지로, 일반 종목보다 참가자 풀이 좁고 경쟁이 굉장히 치열합니다.

지표	결과	비고
글로벌 순위	175위 / 6,332명	상위 2.8%
한국 순위	1위	
CodinPoints	3,962 / 5,000	만점 대비 79.2%
대회 형식	Summer Challenge 2024 (연례 챌린지)	
평가 방식	봇 간 직접 대결 매치메이킹 + ELO	객관적·정량적 평가

6.2 실행시간 성능 특성

단계	시간 예산	봇 동작
첫 턴	1,000ms	메달 가치 초기화 + 전 게임 상태 파싱
일반 턴	100ms	깊이 12 Expectimax + α - β 가지치기
시간 측정	chrono::high_resolution_clock (L546)	매 턴 종료 시 stderr로 소요시간 로깅

6.3 정량적 최적화 효과

최적화 기법	단축 효과
Log-기댓값 변환	곱셈 결합 → 덧셈 결합 → α - β 가지치기 적용 가능
무브 오더링 (1-ply 사전 평가)	α - β 가지치기 효율 2~3배 향상 (좋은 수 우선 평가)
빔 폭 동적 조정 (4→3→2)	깊이 12에서 분기 폭발 차단 — 트리 크기 수십 분의 1
Undo 스택	깊은 복사 회피 → 메모리 할당 0, O(1) 상태 복원
$\sqrt{}$ 호출 제거 (거리3)	양궁 평가 핫루프에서 부동소수점 $\sqrt{0}$ 회
SIMD/AVX2	핫루프 자동 벡터화

6.4 달성 목표 검증

2.3에서 정의했던 개발 목표 전부 달성했습니다.

- 100ms 안에 깊이 12 게임 트리 탐색 수행
- 동시 행동 게임의 확률적 상대 모델링 (Sigmoid)
- 곱셈 목적함수의 α - β 가지치기 적용 가능 (Log 변환)
- 4개 미니게임 각각에 도메인 특화 평가 함수 적용
- 글로벌 상위권 진입 (Top 2.8%)
- 한국 내 최고 성능 (1 위)

7. 현대 AI 시스템과의 대응 관계

이 AI 봇은 딥러닝 기반이 아니라 고전 AI 탐색(Search-based AI) 계열입니다. 그렇지만 현대 강화학습 및 LLM 시스템의 핵심 골격을 그대로 공유하고 있다고 생각합니다.

본 AI 봇 기법	대응되는 현대 AI 시스템
Expectimax + α - β	게임 AI의 표준 트리 탐색 (체스, 포커 AI Libratus/Pluribus의 골격)
Sigmoid/Boltzmann 정책	강화학습 Soft Actor-Critic(SAC)의 Soft Policy / 엔트로피 정규화
온도 파라미터 튜닝	LLM 디코딩의 temperature 파라미터 / SAC의 entropy temperature
Log-기댓값 최대화	Policy Gradient의 log-likelihood objective / NLL 손실
빔 서치 (4-min(2,d/3))	LLM 디코딩의 Beam Search / Seq2Seq 추론
시간 감쇠 보상	강화학습의 할인계수(γ) / Reward Shaping
휴리스틱 평가함수	알파고 Value Network의 hand-crafted 전조
다목적 → Log 합산	다목적 RL의 Geometric Mean Reward

본 AI 봇은 신경망 학습 없이도 현대 강화학습·LLM의 핵심 기법(Soft Policy, Temperature, Beam Search, Reward Shaping, Geometric Mean Reward)을 직접 설계·구현한 결과물, 특히

Sigmoid 기반 상대 모델링은 SAC의 Soft Policy와 정확히 동치이고, 신경망 학습 없이도 비례적 합리성을 모델링했다는 점에서 의미가 있다고 생각합니다.

8. 결론

Olympbits AI Bot의 핵심 동작과 원리, 성능 결과를 정리하면 다음과 같습니다.

1. 복합 동시-다목적 환경의 AI 설계 능력: 4 개 게임이 동시에 진행되고 3 명이 동시에 행동하는 가장 복잡한 류의 봇 게임에서, 단일 통합 의사결정 시스템 구축.
2. 확률적 상대 모델링 능력: Sigmoid/Boltzmann 정책으로 RL 의 Soft Policy 개념을 직접 구현, 결정론적 상대 가정의 한계를 극복.
3. 다목적 → 단일목적 변환의 수학적 통찰: 메달 곱셈 구조를 Log 변환으로 합산화 → α - β 가지치기 적용 + 기하평균 최적화 효과 동시 달성.
4. 하이퍼파라미터 도메인 튜닝: 4 개 게임 각각에 대한 온도·평가함수 별도 설계 및 튜닝.
5. 실시간 추론 최적화: 100ms 제약 안에서 깊이 12 트리 탐색을 위해 무브 오더링, 빔 서치, Undo 스택, SIMD 등 다층적 최적화.
6. 객관적 검증 통과: 글로벌 6,332 명과 직접 대결해 글로벌 Top 2.8%, 한국 1 위 달성.

8.1 향후 확장 가능성

- 시그모이드 정책 → **Policy Network(다음 수 추천)**으로 교체 → 강화학습 자가대국(Self-play)으로 발전 가능
- 게임별 휴리스틱 평가함수 → **Value Network(형세평가)**로 대체 → AlphaZero 골격 완성
- 온도 파라미터 → 강화학습으로 자동 학습
- **다목적 RL 프레임워크(MORL)**로의 직접 확장 — 본 봇의 Log-기댓값 변환은 이미 MORL 의 표준 기법

9. 첨부

파일	내용
source_ai_bot_olymbits/olymbits.cpp	봇 전체 소스 (C++)